Universidad Politécnica de Pachuca



“PRACTICA 3”

Tecnologías de la Información e Innovación Digital

04_01

Nombre de la Alumna: Ordaz Cano Mariana

Docente: Montoya Cerro Miguel Ángel

Pachuca de Soto, Hgo. a 09 de Octubre del 2026

1. Índices

Índice de páginas

1. Índices 2
2. Introducción 2
3. Marco Teórico 3
4. Desarrollo 4
5. Conclusiones 10
6. Anexos 11
7. Referencias 11

Índice de ilustraciones

Ilustración 1: Creación e implementación inicial del contenedor Row 5
Ilustración 2: Adición de textos iniciales con nombre y apellidos 5
Ilustración 3: Aplicación de horizontalArrangement = Arrangement.Center 6
Ilustración 4: Expansión del ancho con Modifier.fillMaxWidth() 6
Ilustración 5; Uso de Arrangement.SpaceBetween para distanciar textos 7
Ilustración 6: Separación del nombre y apellidos en componentes Text independientes 7
Ilustración 7: Aplicación de Modifier.padding(16.dp) para espacio superior 8
Ilustración 8: Duplicación de elementos provocando desbordamiento en Row 8
Ilustración 9: Migración y estructuración de la interfaz hacia LazyRow 9
Ilustración 10: Código final organizado del componente LazyRow 9

Índice de tablas

Tabla 1 Comparativa entre Row y LazyRow 10

2. Introducción

1.1 Contextualización

En el desarrollo de aplicaciones móviles modernas para la plataforma Android con Jetpack Compose, la disposición horizontal de los componentes visuales resulta indispensable para construir menús, tarjetas, barras de herramientas y galerías de elementos. Jetpack Compose

ofrece herramientas declarativas eficientes como Row y LazyRow que permiten organizar elementos en filas de manera intuitiva utilizando Kotlin.

1.2 Planteamiento del problema u objeto de estudio

Al diseñar interfaces de usuario horizontales, los componentes pueden sobrepasar los límites físicos de la pantalla si no cuentan con un contenedor que administre el espacio o permita el desplazamiento (scroll). En esta práctica se analiza la diferencia práctica entre un contenedor estático Row y un contenedor dinámico LazyRow, explorando opciones de alineación horizontal y distribución espacial.

1.3 Objetivo general

Desarrollar una interfaz en Android Studio empleando Jetpack Compose para implementar la organización horizontal de componentes mediante Row y LazyRow, aplicando modificadores de ancho, distribución de espacio y márgenes para lograr un diseño responsivo y deslizable.

1.4 Objetivos específicos

- Implementar un contenedor Row para organizar elementos en sentido horizontal.
- Aplicar modificadores como fillMaxWidth() y padding() para controlar el tamaño y los espacios entre elementos.
- Utilizar propiedades de alineación como Arrangement.Center y Arrangement.SpaceBetween.
- Identificar las limitaciones del contenedor Row ante la acumulación excesiva de elementos visuales.
- Migrar la estructura a LazyRow para habilitar el desplazamiento horizontal fluido.
- Validar el funcionamiento de la interfaz mediante la ejecución en el emulador de Android Studio.

1.5 Justificación

Dominar la estructuración horizontal a través de Row y LazyRow es primordial para los desarrolladores Android, ya que facilita la creación de carruseles, listas horizontales de categorías y componentes complejos, garantizando que la aplicación se adapte correctamente a múltiples resoluciones de pantalla.

3. Marco Teórico

2.1 Row en Jetpack Compose

Row es un contenedor composable de diseño que posiciona a sus elementos secundarios de manera secuencial a lo largo de un eje horizontal (de izquierda a derecha).

2.2 LazyRow

LazyRow es un contenedor composable para listas horizontales con soporte de desplazamiento. A diferencia de Row, solo compone y renderiza los elementos que actualmente son visibles en la pantalla, optimizando el rendimiento y el uso de memoria.

2.3 HorizontalArrangement

Es un parámetro que define cómo se distribuyen los elementos secundarios a lo largo del eje horizontal dentro de un contenedor Row o LazyRow.

2.4 Arrangement.Center y SpaceBetween

Arrangement.Center agrupa y ubica todos los elementos secundarios en el centro horizontal del contenedor. Por su parte, Arrangement.SpaceBetween distribuye el espacio horizontal restante de manera equitativa entre los elementos, colocándolos a lo largo de todo el ancho disponible.

2.5 Modificadores fillMaxWidth y Padding

El modificador fillMaxWidth() indica al componente que debe expandirse hasta ocupar todo el ancho disponible concedido por el contenedor padre. El modificador padding() agrega márgenes o espacio libre en los bordes del componente para evitar colisiones visuales con otros elementos.

4. Desarrollo

3.1 Creación e implementación de un contenedor Row

Para iniciar con la práctica, debajo de los bloques de texto utilizados en la práctica anterior, se procedió a declarar e instanciar la etiqueta composable Row dentro del archivo principal del proyecto para comenzar a trabajar con filas horizontales.

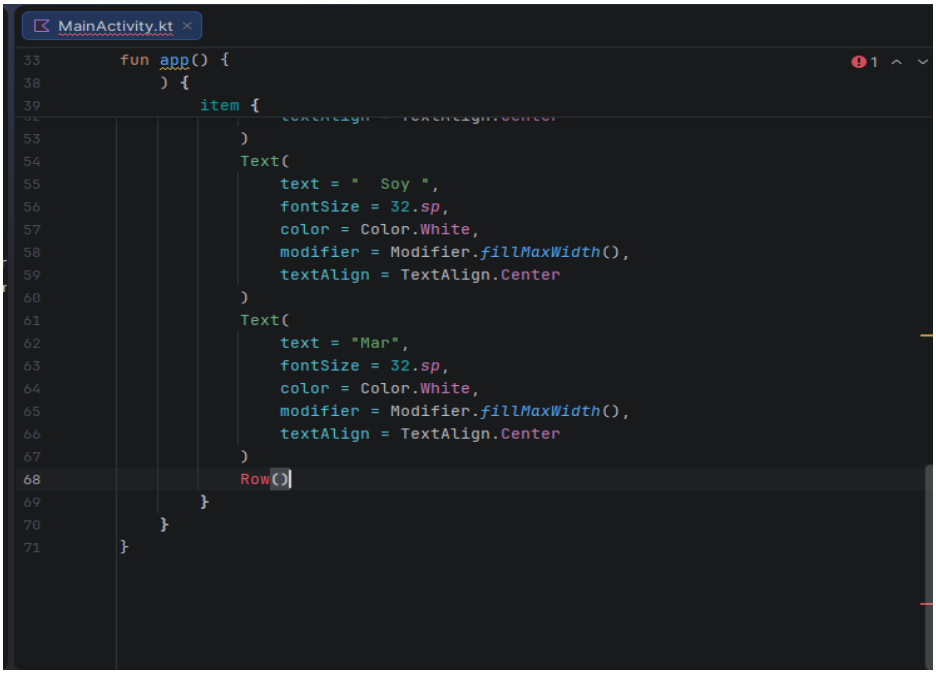


Ilustración 1: Creación e implementación inicial del contenedor Row

3.2 Inclusión de elementos de texto iniciales en la fila

Dentro del bloque del Row se agregaron componentes de tipo Text para colocar mensajes con nombres y apellidos, permitiendo comprobar cómo se distribuyen horizontalmente uno al lado del otro.

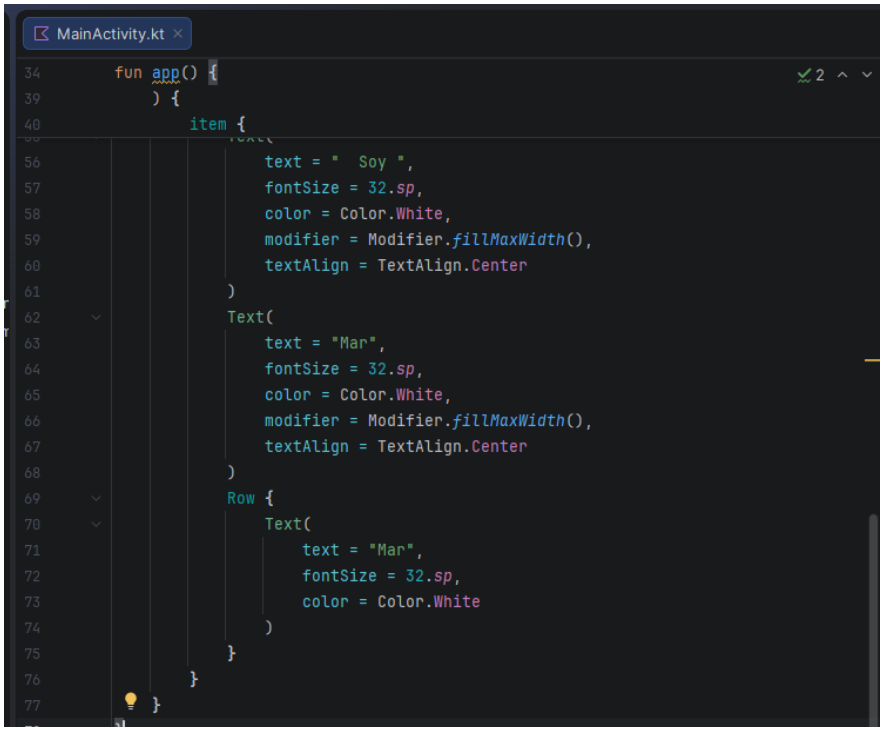


Ilustración 2: Adición de textos iniciales con nombre y apellidos

3.3 Alineación horizontal mediante Arrangement.Center

Con el fin de posicionar el contenido en la zona central de la pantalla, se agregó el parámetro horizontalArrangement = Arrangement.Center a la definición del Row.

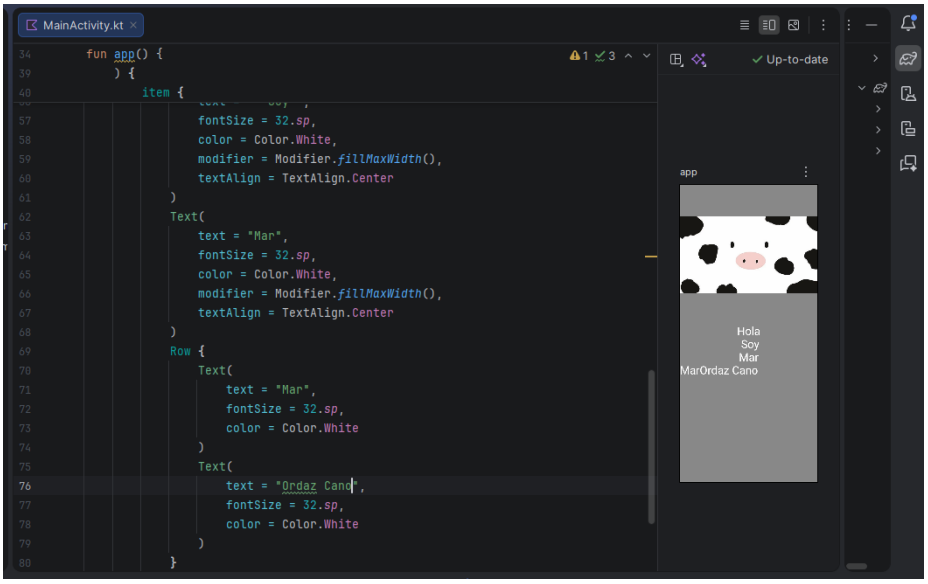


Ilustración 3: Aplicación de horizontalArrangement = Arrangement.Center

3.4 Expansión del ancho con Modifier.fillMaxWidth()

Debido a que el Row inicialmente solo ocupaba el ancho de sus elementos internos y no se centraba visualmente en la pantalla, se agregó el modificador modifier = Modifier.fillMaxWidth() para obligar a la fila a extenderse a lo largo de todo el renglón horizontal.

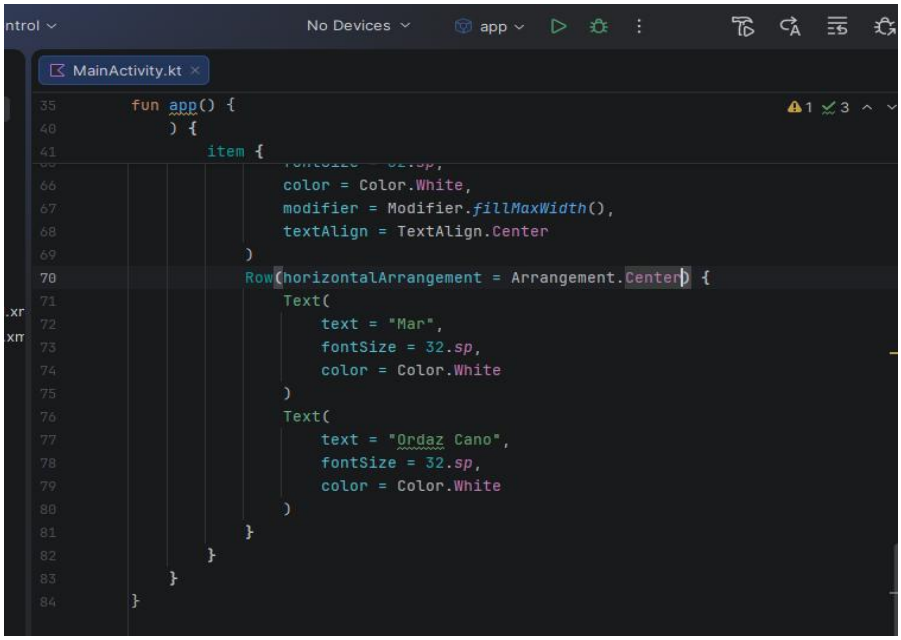


Ilustración 4: Expansión del ancho con Modifier.fillMaxWidth()

3.5 Distribución equitativa de elementos con Arrangement.SpaceBetween

Posteriormente, se cambió el valor de centrado por horizontalArrangement = Arrangement.SpaceBetween. Esta propiedad permitió empujar los elementos hacia los extremos opuestos de la pantalla, distribuyendo todo el espacio disponible entre ellos.

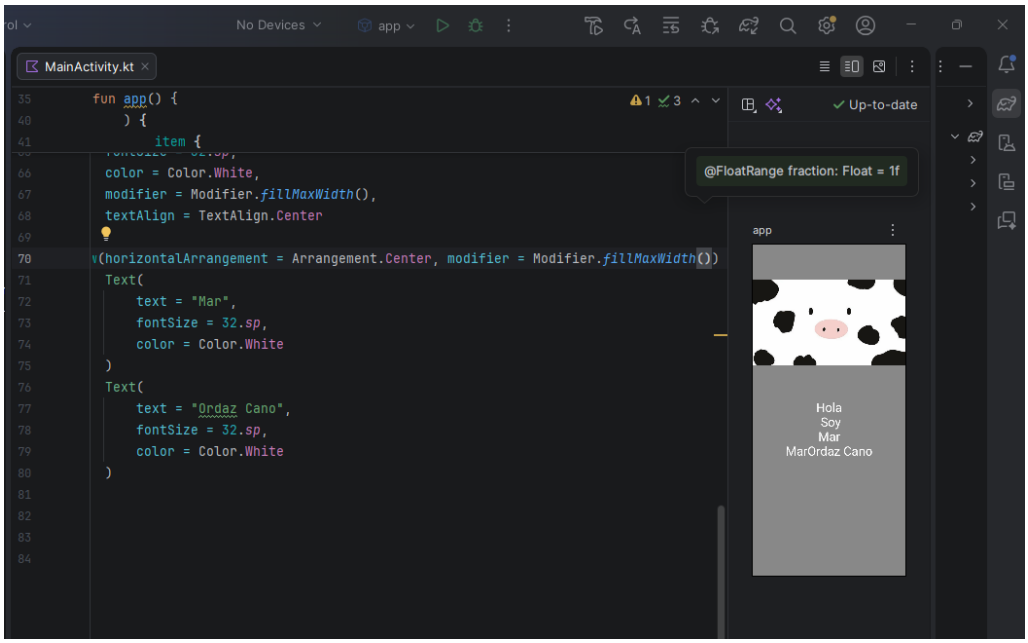


Ilustración 5; Uso de Arrangement.SpaceBetween para distanciar textos

3.6 Separación de nombres y apellidos en componentes independientes

Para observar con mayor detalle el comportamiento de SpaceBetween, se dividió el texto completo en componentes Text individuales para el nombre y cada uno de los apellidos. Al actualizarse la vista previa, se apreció cómo cada palabra se separó automáticamente manteniendo un espacio uniforme entre ellas.

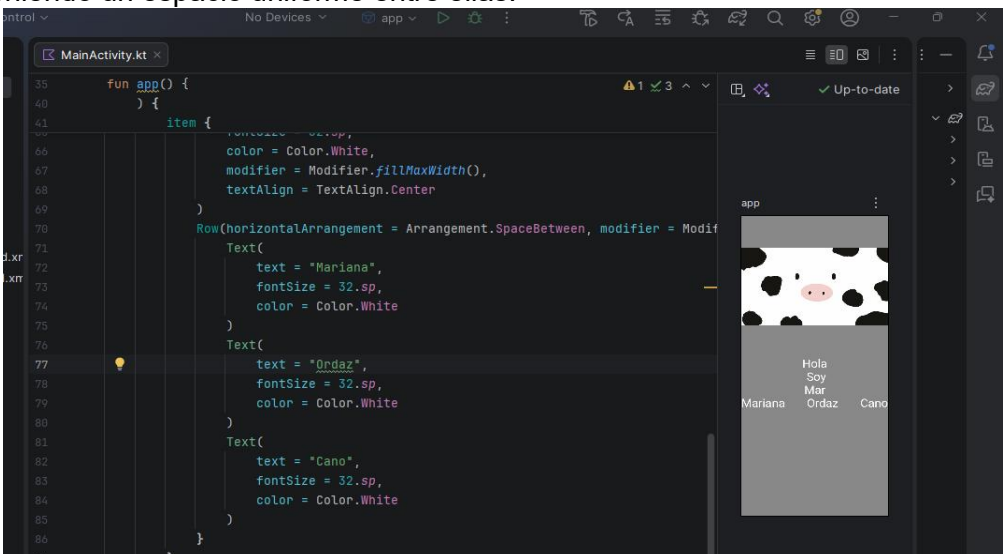


Ilustración 6: Separación del nombre y apellidos en componentes Text independientes

3.7 Aplicación del modificador Padding para evitar sobreposición

Para evitar que la nueva fila quedara totalmente pegada al contenido previo en la parte superior, se encadenó el modificador .padding(top = 16.dp) (o `padding(16.dp)`), generando una separación de margen adecuada y limpia.

```
72              LazyRow(
73                  horizontalArrangement = Arrangement.SpaceBetween,
74                  modifier = Modifier
75                      .fillMaxWidth()
76                      .padding( all = 16.dp)
77              ) {
```

Ilustración 7: Aplicación de Modifier.padding(16.dp) para espacio superior

3.8 Duplicación de elementos para evaluar saturación horizontal

Con la finalidad de analizar las limitaciones de un Row estándar, se duplicaron y pegaron los bloques de texto múltiples veces de forma consecutiva. Como resultado, el contenido rebasó el ancho físico de la pantalla, provocando que los elementos quedaran amontonados y recortados sin posibilidad de desplazarse.

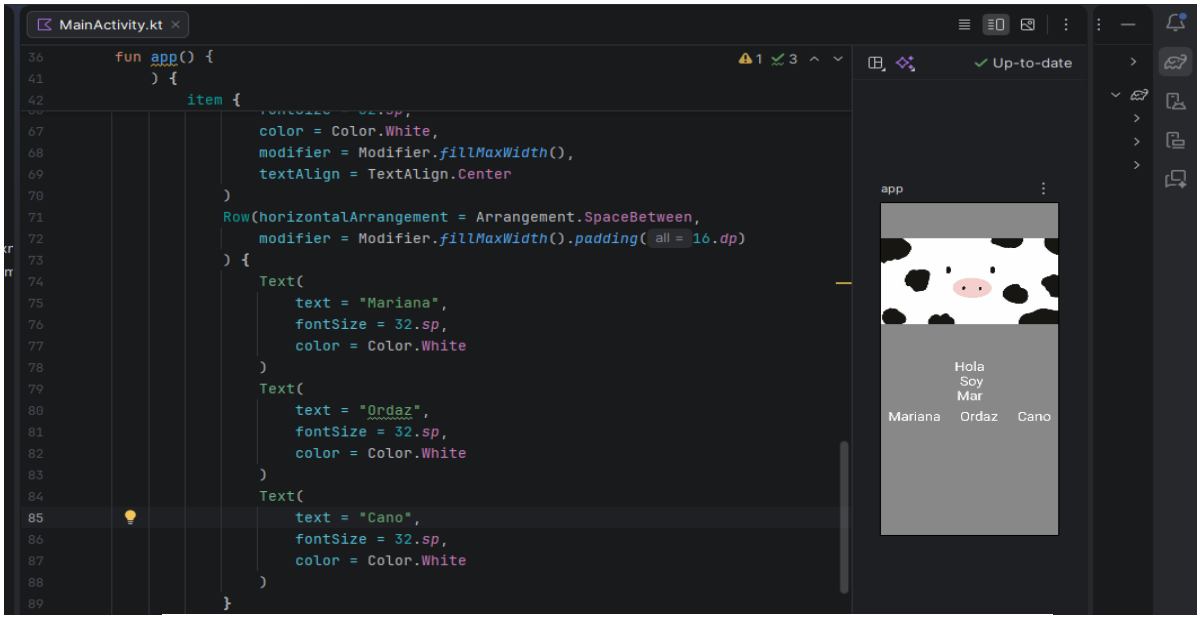


Ilustración 8: Duplicación de elementos provocando desbordamiento en Row

3.9 Implementación de LazyRow para desplazamiento horizontal

Para solucionar el problema de desbordamiento, se reemplazó la etiqueta Row por un contenedor LazyRow. Esta modificación permitió organizar de forma dinámica todos los elementos y habilitar automáticamente el desplazamiento (scroll) horizontal de la lista.

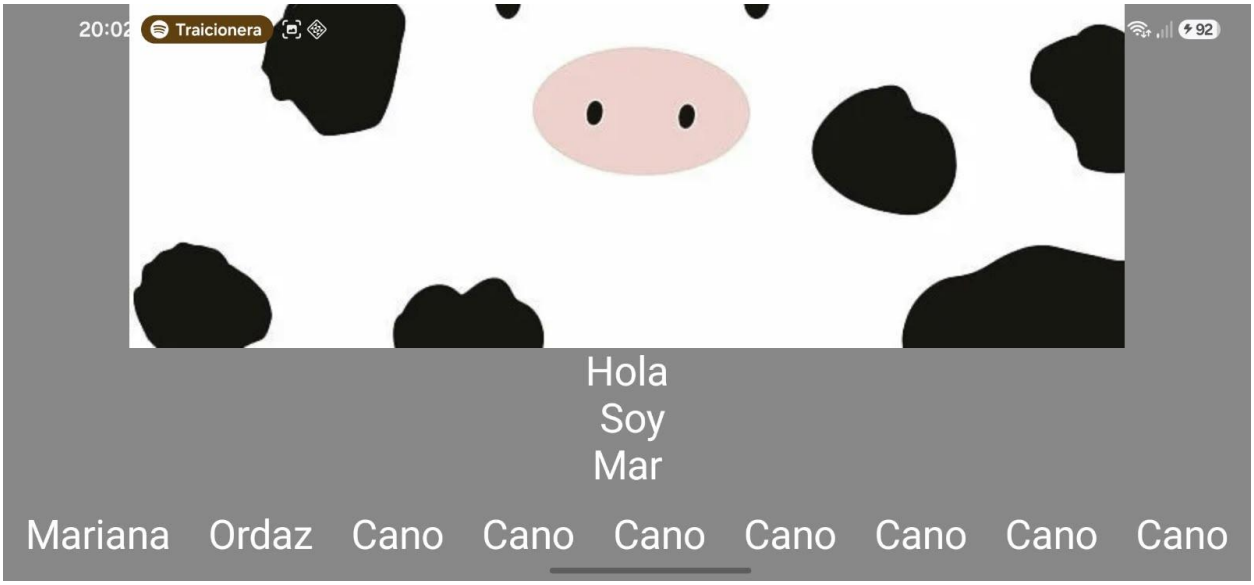


Ilustración 9: Migración y estructuración de la interfaz hacia LazyRow

3.10 Código final organizado y estructurado

Se presenta la estructura de código limpia en MainActivity.kt donde se integran las funciones composables, modificadores y la declaración correcta del LazyRow.

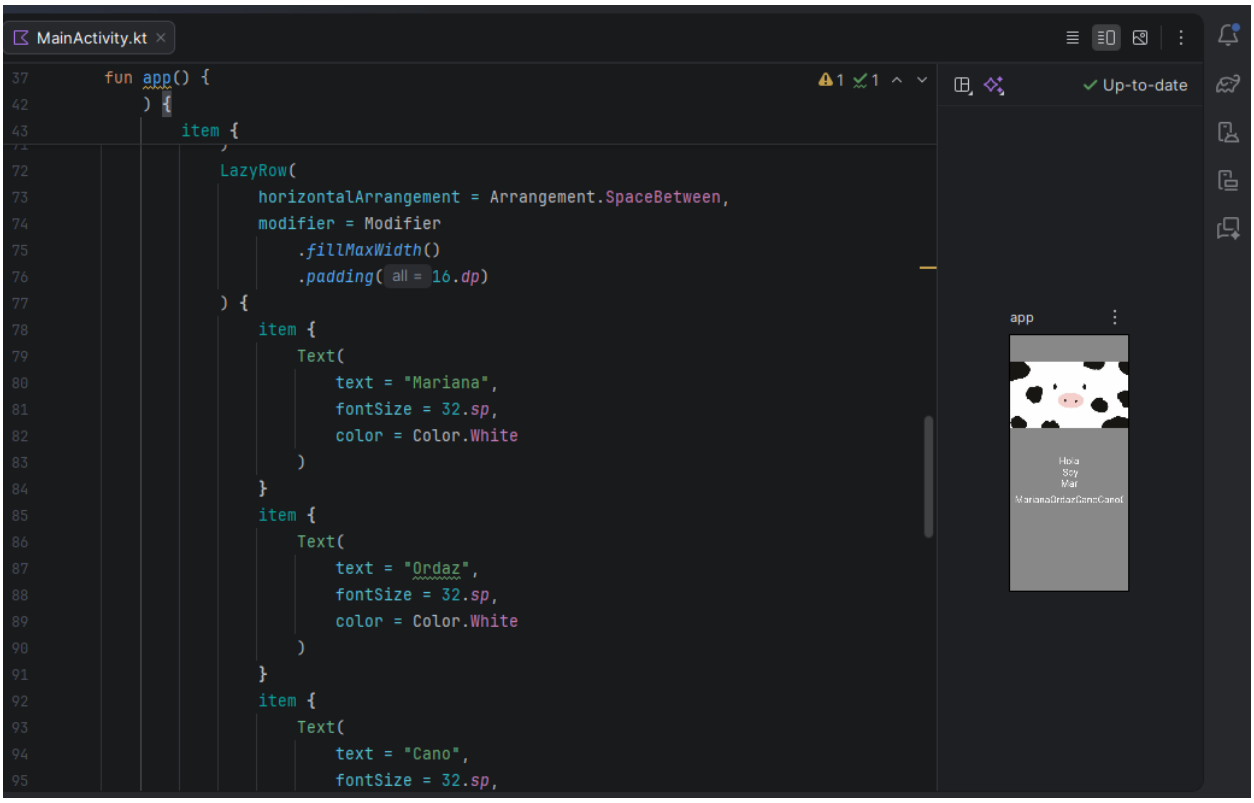


Ilustración 10: Código final organizado del componente LazyRow

3.11 Comparativa técnica entre Row y LazyRow

Tabla 1 Comparativa entre Row y LazyRow

Características	Row	LazyRow
Carga en memoria	Carga e instancia todos los elementos horizontales simultáneamente.	Carga progresivamente solo los elementos visibles en pantalla.
Soporte de Scroll	No tiene desplazamiento nativo (requiere modificadores adicionales).	Posee desplazamiento (scroll) horizontal nativo e implícito.
Rendimiento	Óptimo únicamente para una cantidad pequeña y fija de elementos.	Alta eficiencia y rendimiento optimizado para listas extensas.

5. Conclusiones

4.1 Aportes y limitaciones

Aportes: La realización de esta práctica permitió comprender el funcionamiento de los arreglos horizontales en Jetpack Compose, aprendiendo a manipular las propiedades de alineación como Arrangement.SpaceBetween y comprendiendo la importancia de utilizar LazyRow cuando se manejan secuencias anchas o dinámicas de información.

Limitaciones: En este ejercicio solo se trabajó con componentes de texto plano dentro de la fila horizontal. En escenarios reales de desarrollo, suele requerirse combinar LazyRow con tarjetas personalizadas (Cards), imágenes u objetos dinámicos provenientes de un modelo de datos.

4.2 Recomendaciones y proyecciones

Se sugiere continuar ejercitando la combinación de filas horizontales (LazyRow) dentro de columnas verticales (LazyColumn) para construir interfaces multinivel complejas (como menús

estilo plataformas de streaming), además de agregar interacciones con eventos de clic en los elementos individuales.

6. Anexos

- https://github.com/marianaordz12/Aplicaciones-Moviles-

7. Referencias

- ---

  AristiDevs. (2021, 12 de agosto). *Curso JETPACK COMPOSE desde CERO - CREA TU PRIMERA APP - Tutorial Android COMPOSE* [Video]. YouTube.
- Google Developers. (2024). *Lists and grids in Jetpack Compose (Row and LazyRow)*. Android Developers.